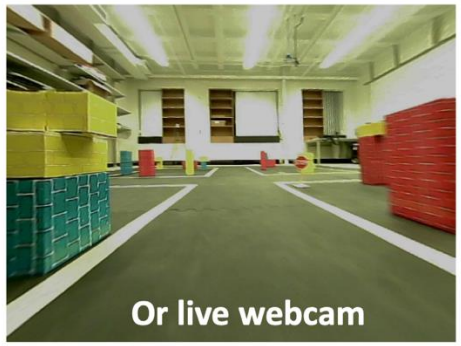
CSC 491/591
Self-Driving Cars: Theory and Practice

Machine Learning and Object Detection (Lab)

Dhruva Ungrupulithaya
Department of Computer Science
NC State University

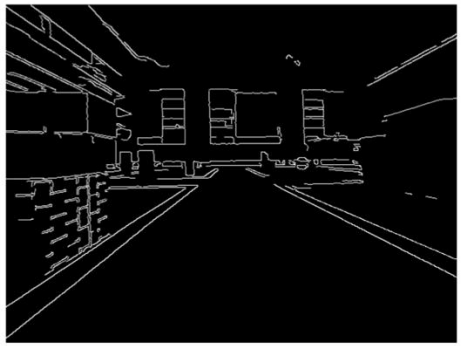
Why do we need Machine Learning?

- Canny edge detection and Hough transform



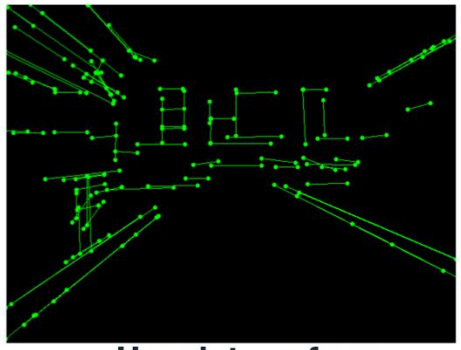
Grayscale

Canny edge detection



Experiment with various values for
`threshold,`
`minLineLength,`
`maxLineGap`

Use `rho=1, theta=np.pi/180`

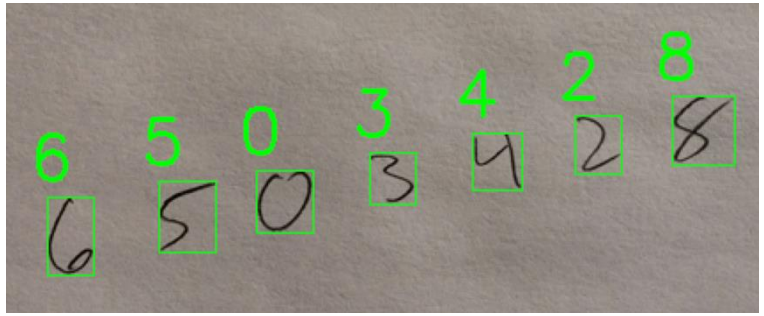


Hough transform

Overlay

What is Machine Learning?

- Learning from data
  - Learn a complex task from large quantities of examples
  - Predict/produce inferences on new data/situation

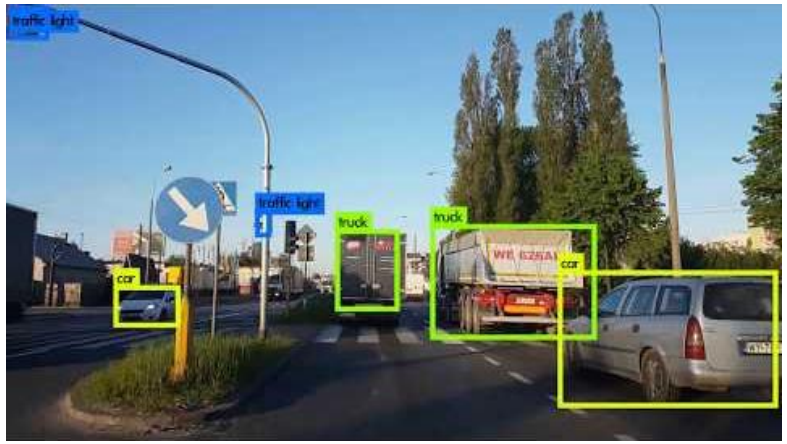


[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

Arthur Samuel, *1959*

A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

Tom Mitchell, *1997*



YOLO (You Only Look Once) Real-Time Object Detection

ML in Autonomous Driving


Truth: 22
Prediction: 22


Truth: 18
Prediction: 18


Truth: 52
Prediction: 37


Truth: 32
Prediction: 32


Truth: 39
Prediction: 39

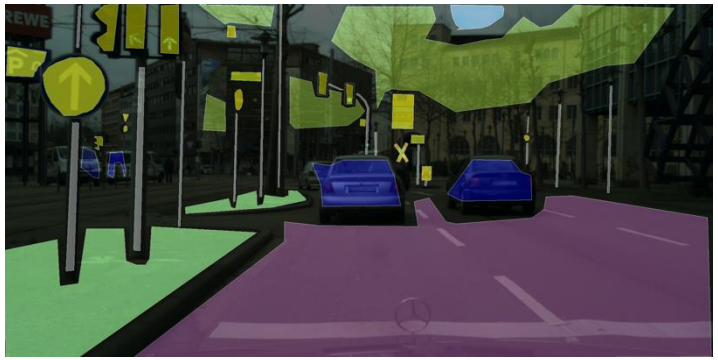

Truth: 61
Prediction: 7

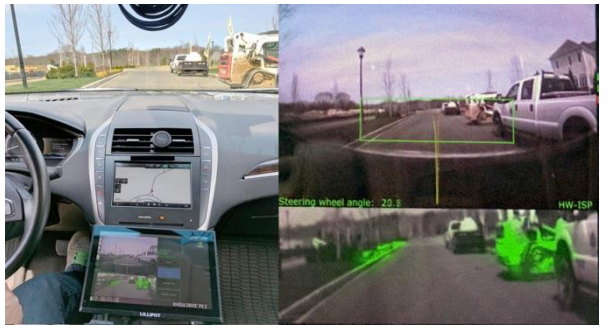

Truth: 22
Prediction: 22

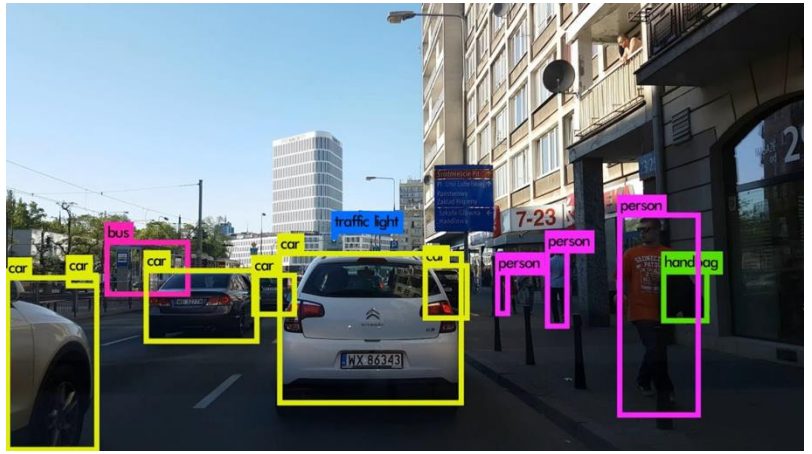

Truth: 17
Prediction: 17


Truth: 32
Prediction: 32

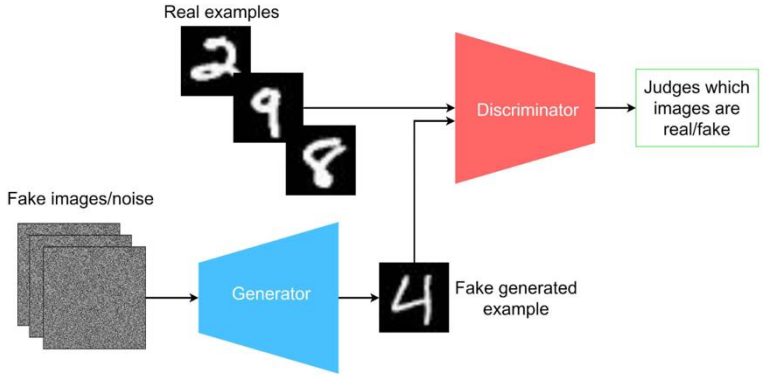

Truth: 53
Prediction: 53



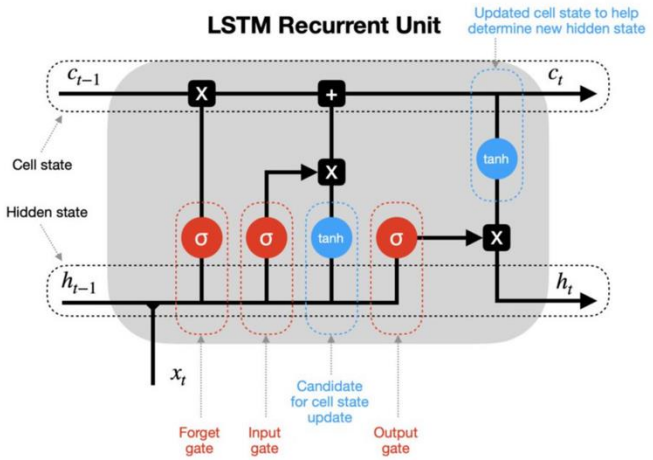




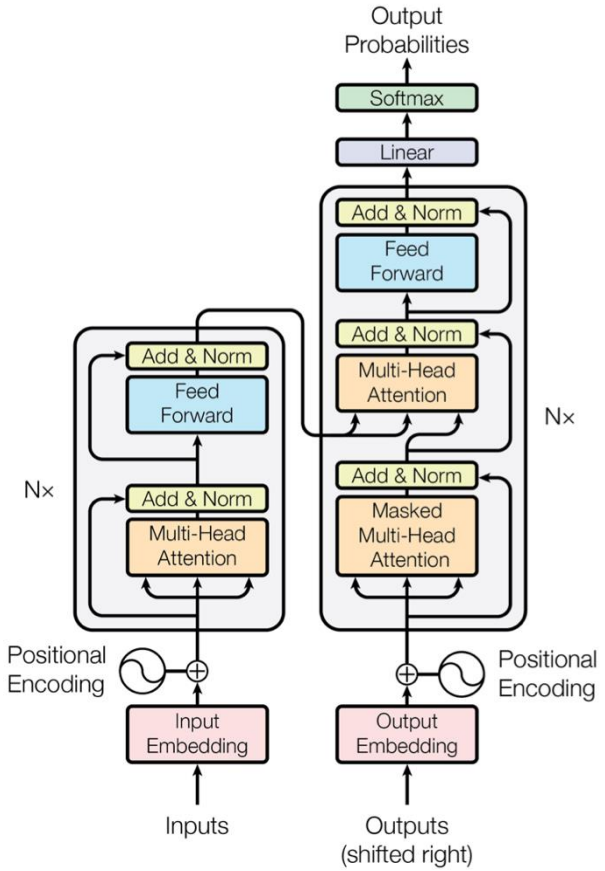
Model Architectures



Generative Adversarial Networks (GANs)



Long-Short Term Memory (LSTM)



Transformers

Convolutional Neural Networks (CNN)

- Most popular architecture for image tasks
- Convolutional (+activation), pooling, convolutional (+activation), pooling, …, fully connected

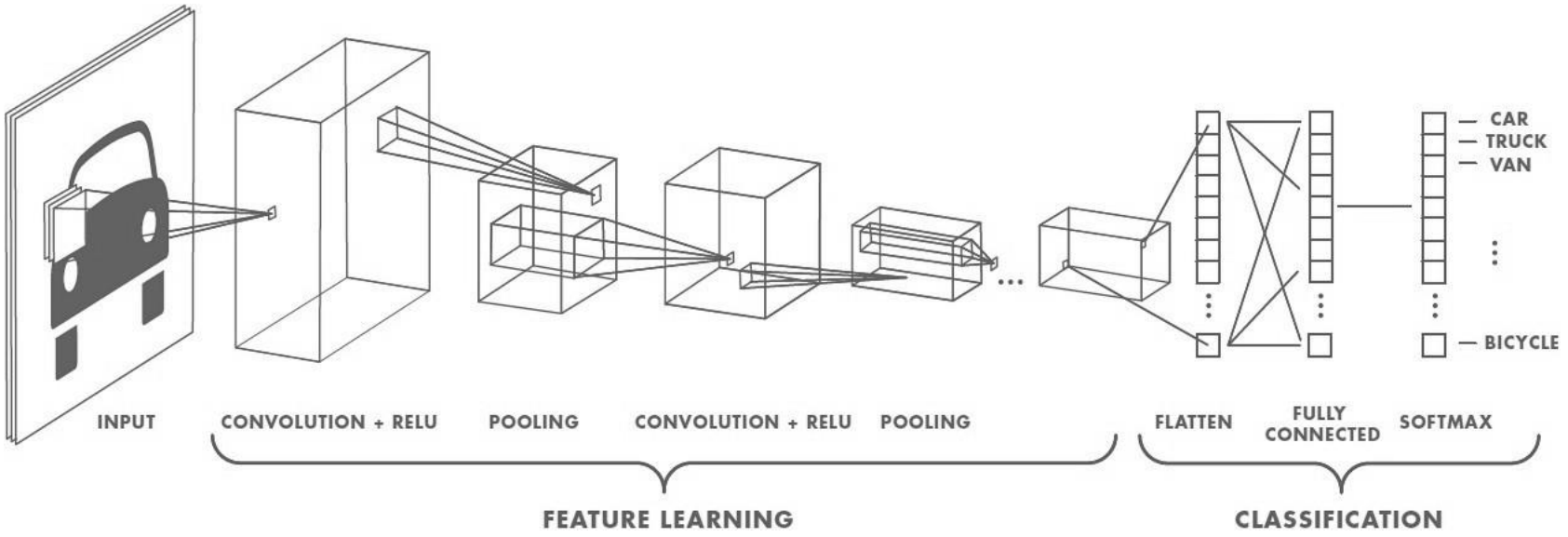
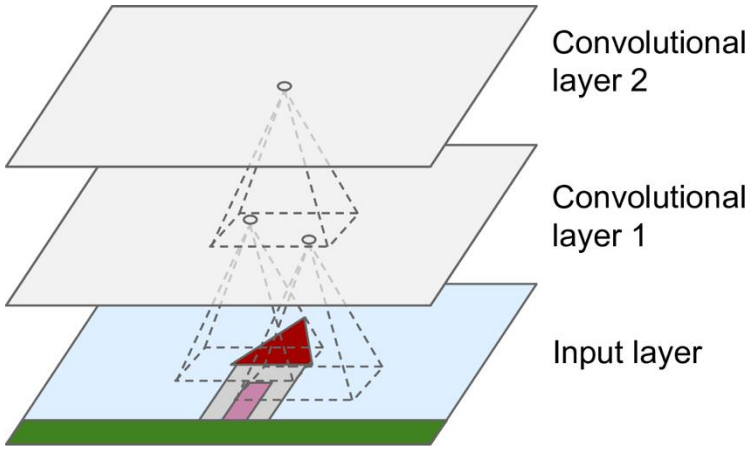


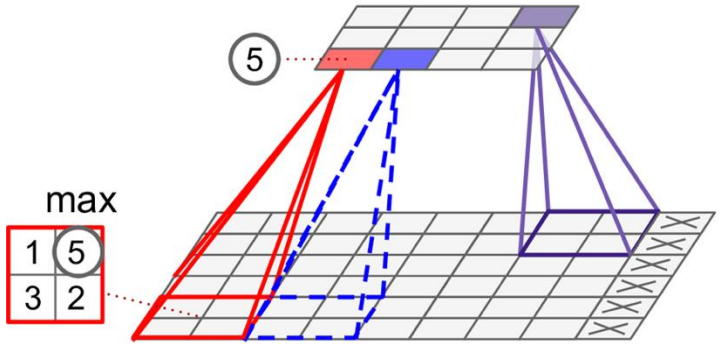
Image source: https://medium.com/intro-to-artificial-intelligence/semantic-segmentation-udaitys-self-driving-car-engineer-nanodegree-c01eb6eaf9d

Basic Building Blocks of CNN

Convolutional layer

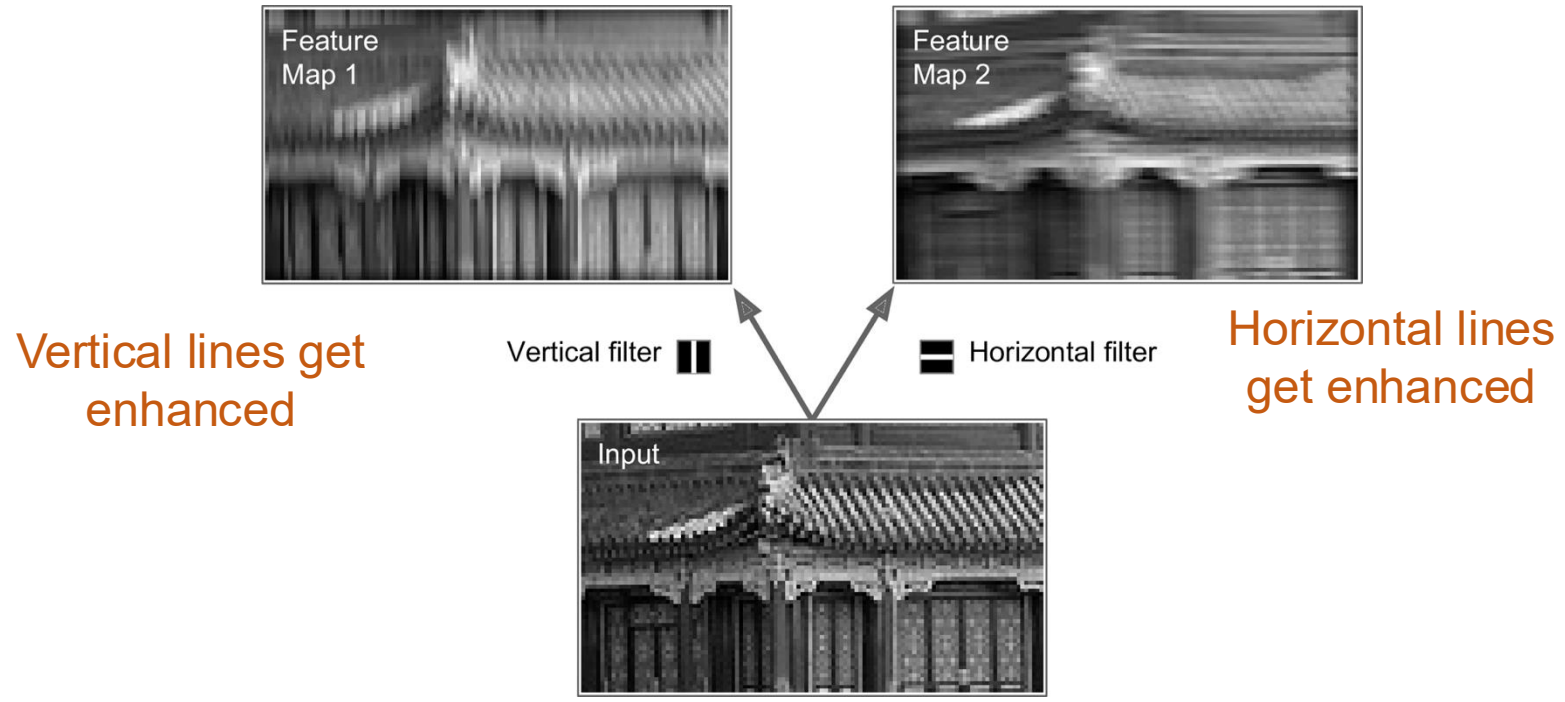


Pooling layer



Convolutional Layer

- Filters (=convolutional kernels)
  - Extracts “features” from the image
  - Complex patterns are found by combining filters
  - CNN training == finding most useful filters (i.e., kernels)



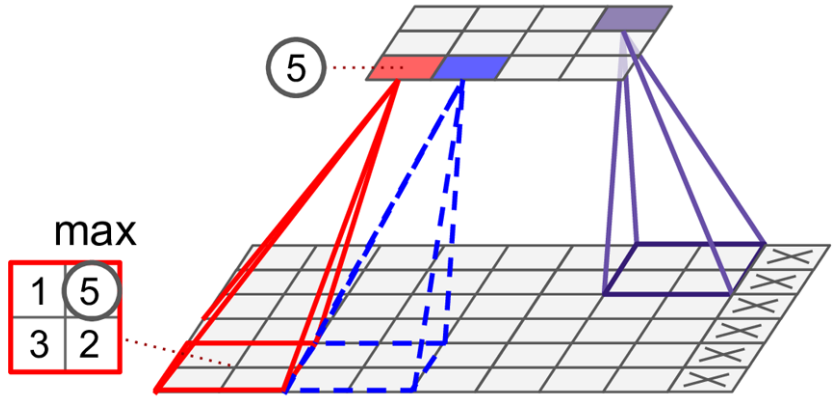
Pooling Layer

- For subsampling (→ reduces dimensionality of feature maps)
  - Aggregates the inputs using an aggregation function such as ‘max’ or ‘mean’
    - Retains the most important information
    - No ‘weights’

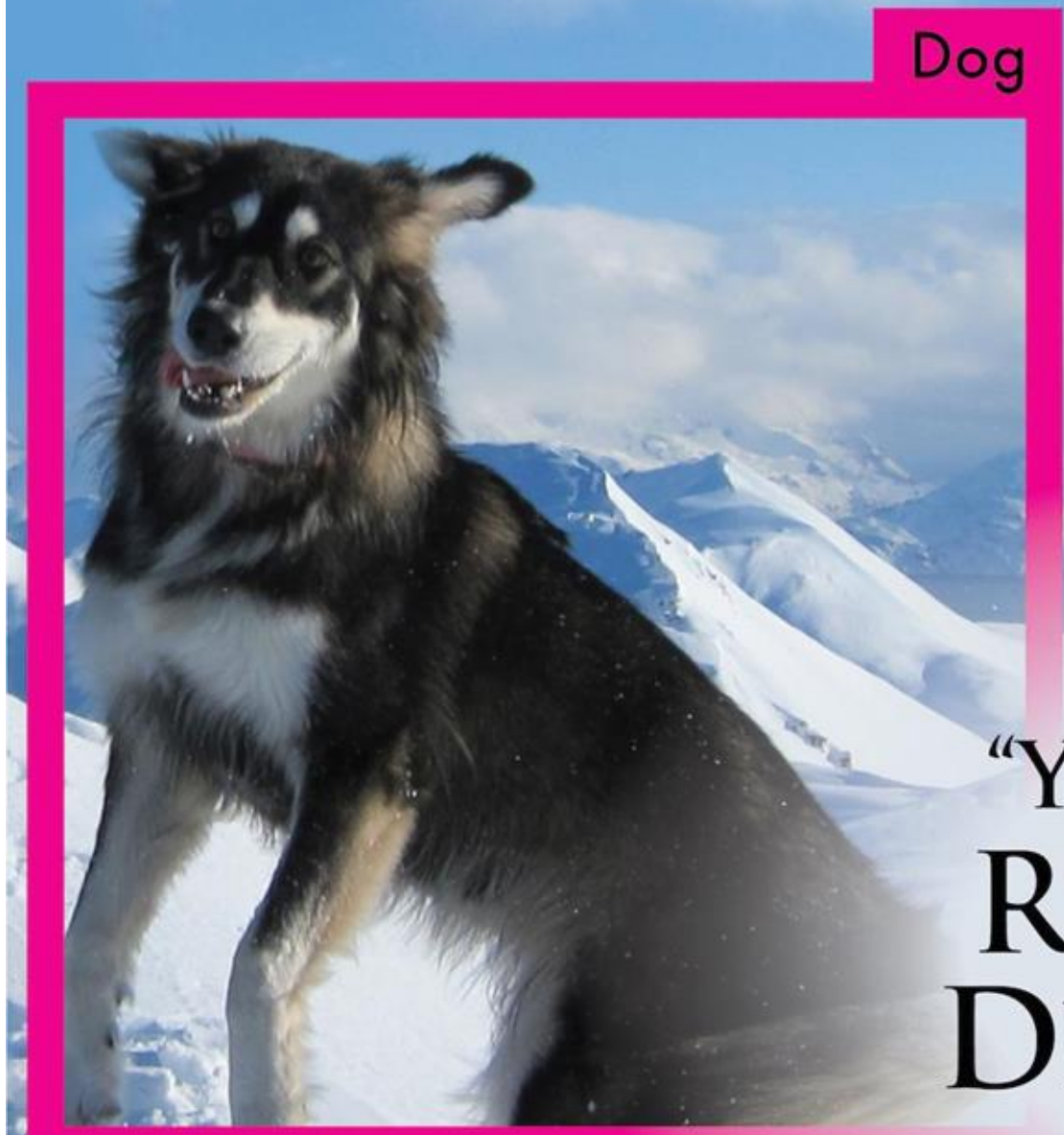
- Example: Max pooling layer
  - Takes the max input value (=strongest feature)

2x2 pooling kernel & Stride of 2

→ Drops ¾ of inputs



Joseph Redmon
Ross Girshick
Santosh Divvala
Ali Farhadi



Most Accurate Real-Time Detector 2016

Fastest Object Detector in the Literature 2016

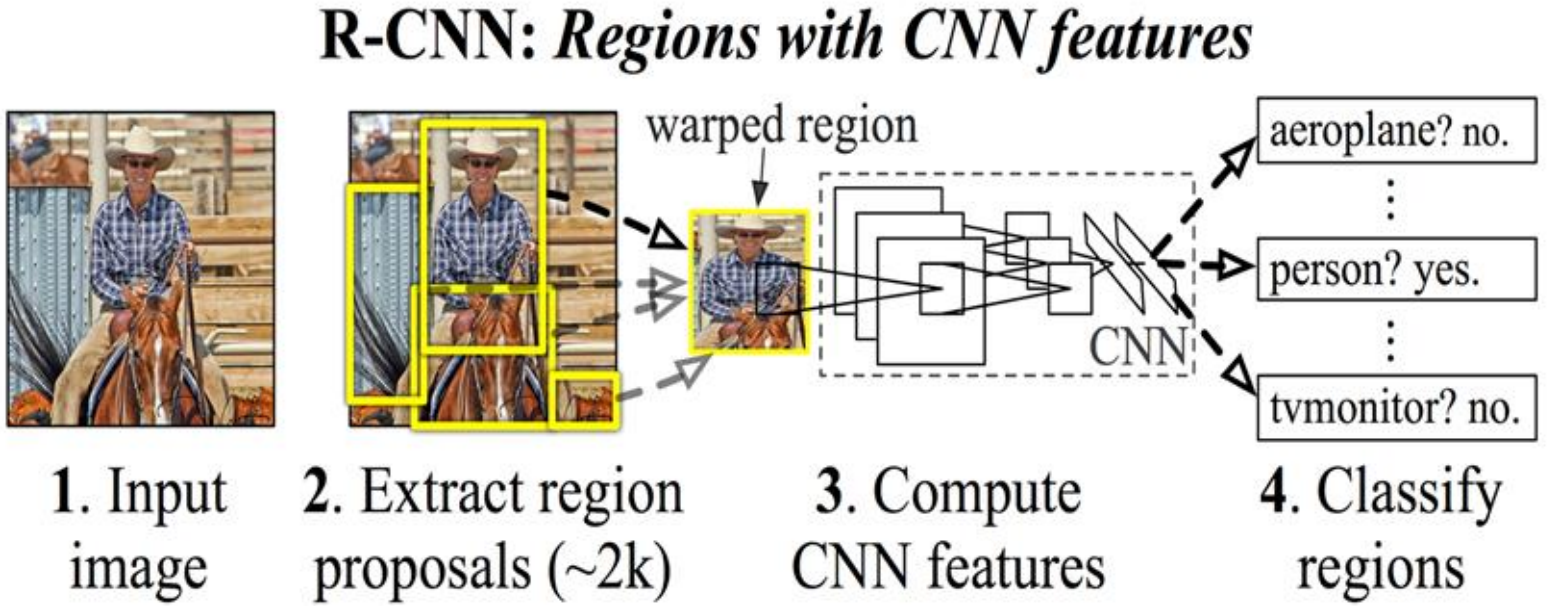
"You Only Look Once"

Real-Time Detection

Author's Resume…

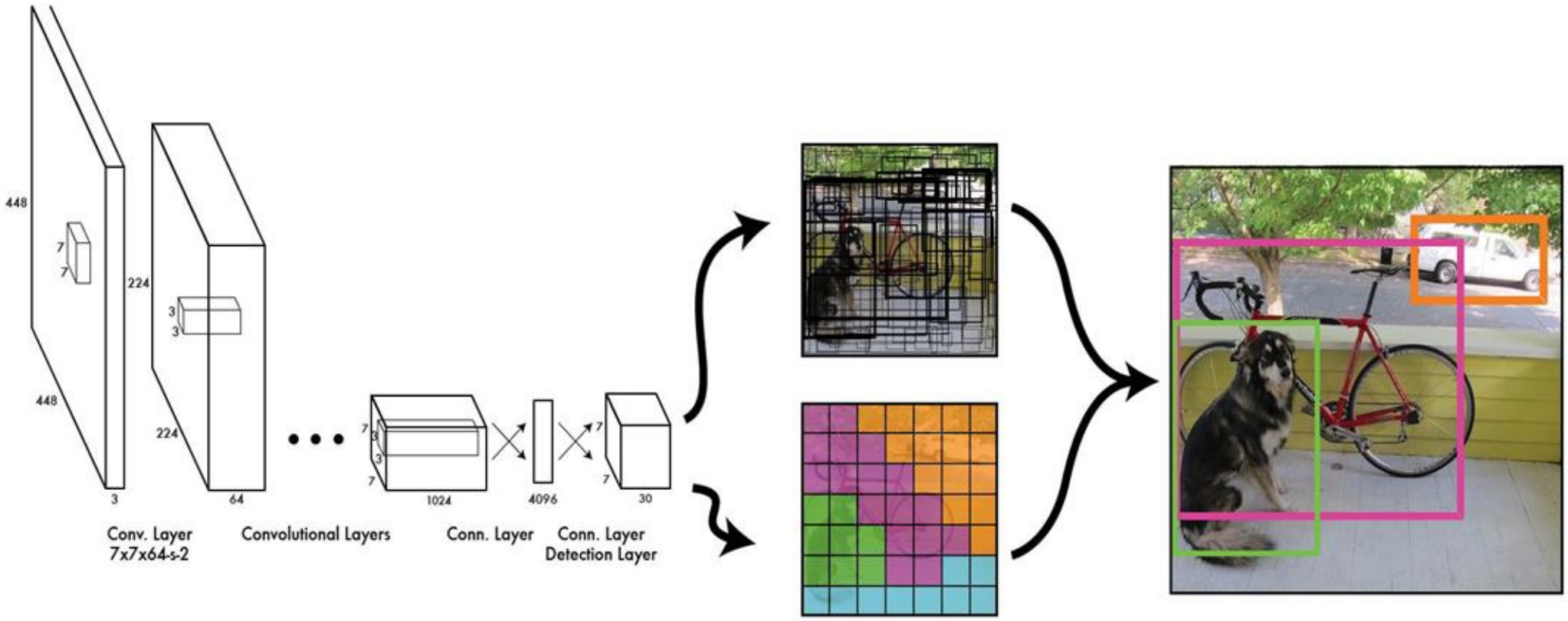
https://pjreddie.com

Recurrent CNNs

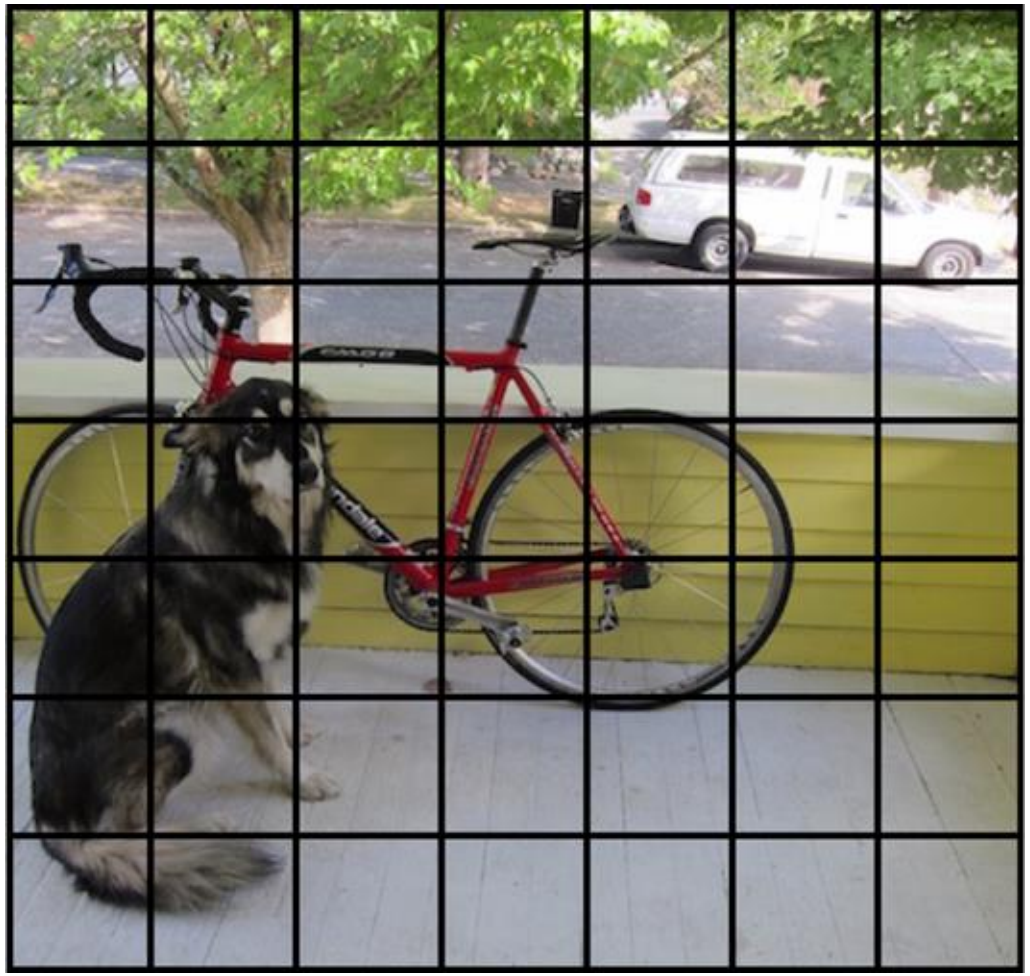


https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Redmon_You_Only_Look_CVPR_2016_paper.pdf

You Only Look Once

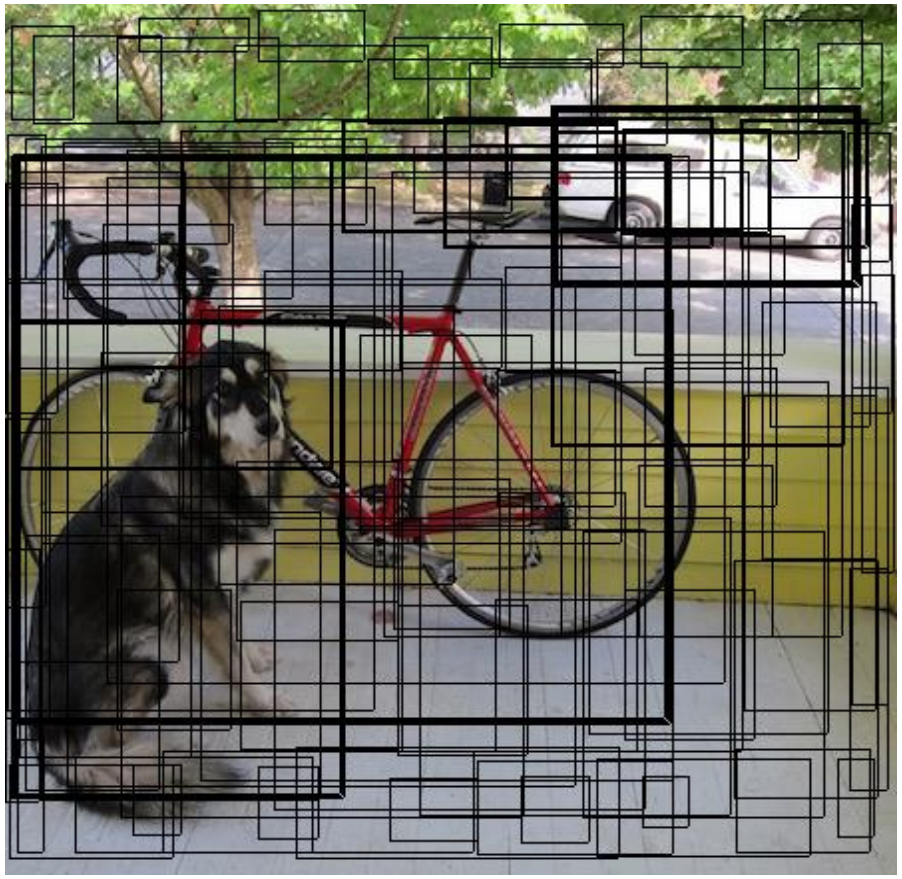


You Only Look Once

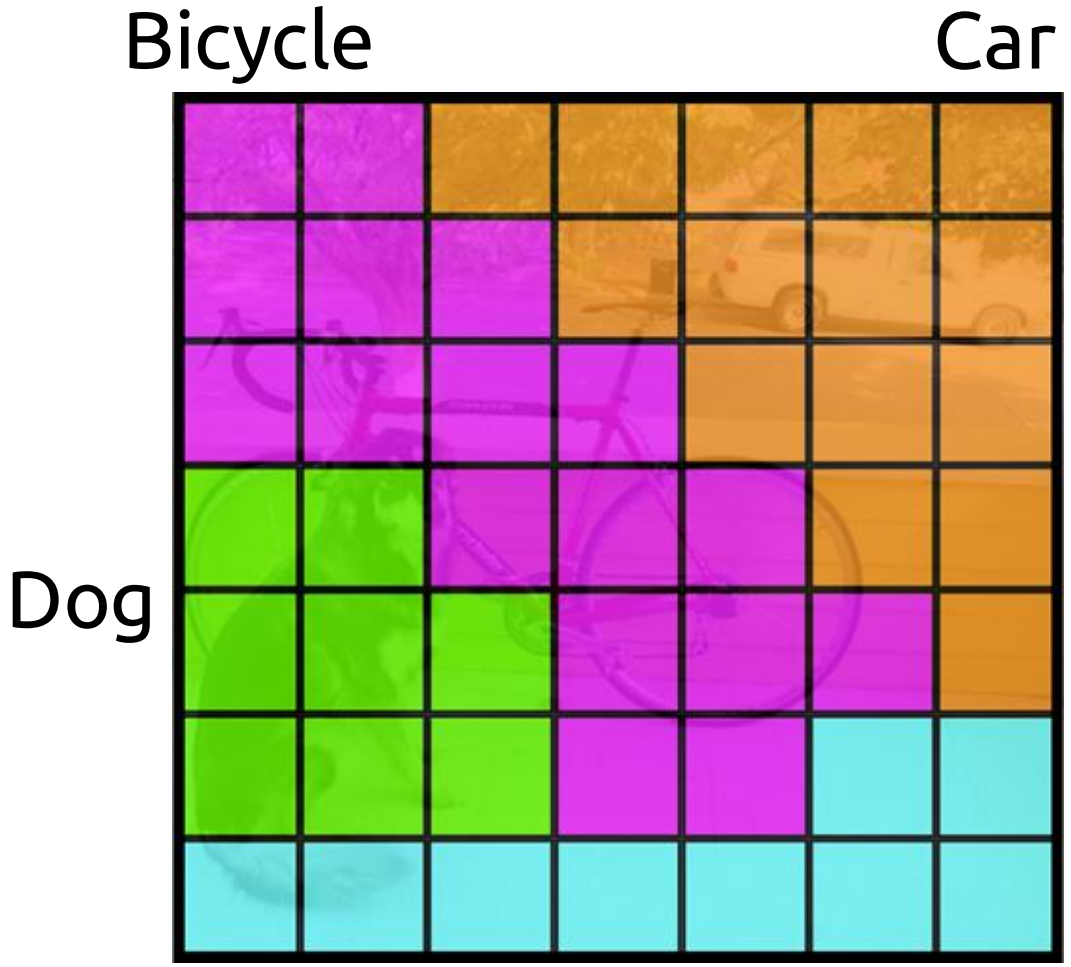


Divide image to 7 x 7 Grid

You Only Look Once

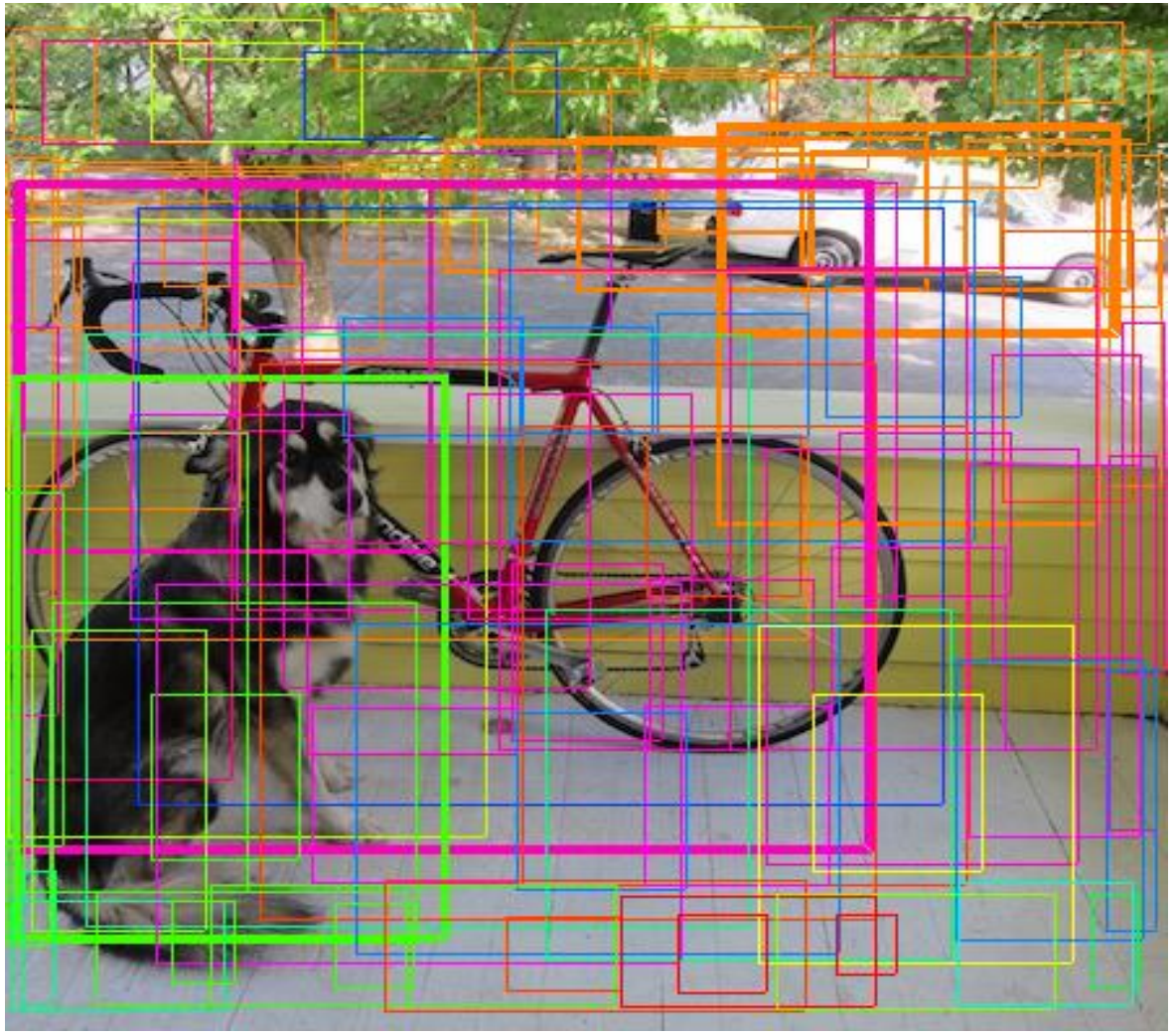


Bounding Boxes + Confidence



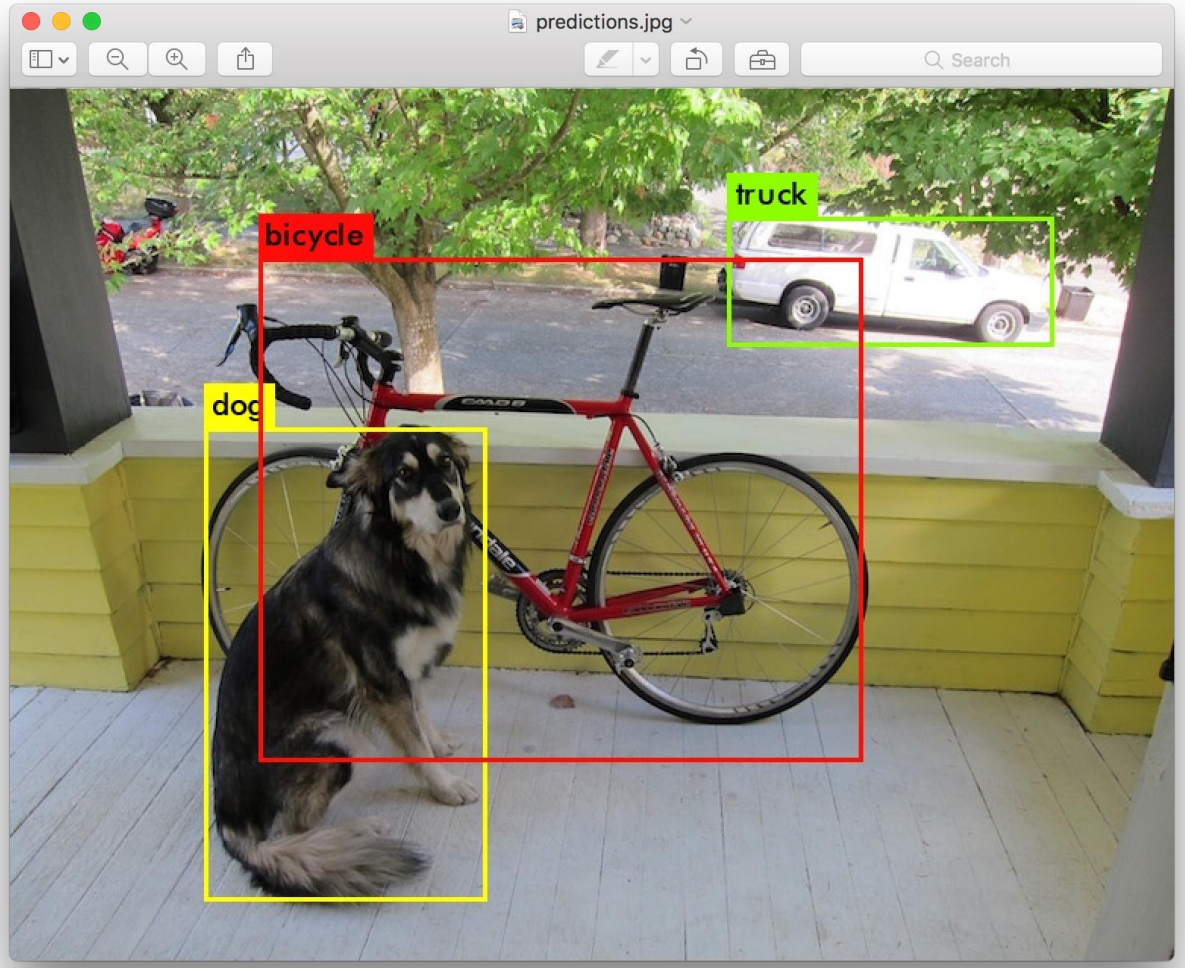
Class Probabilities

You Only Look Once



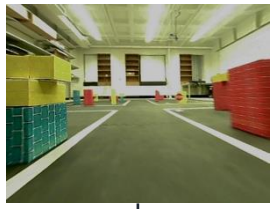
Bounding Boxes + Class Predictions

You Only Look Once



After Filtering and Non-Maximal Suppression

Today's Lab: Stop Sign Detection



Camera input

YOLO Object Detector

Object Class = Stop Sign

Box Size > Threshold

"Stop"

Driver

Yolo Detection Node

YOLO Detector ROS2 Node

- Don’t forget to run Zed ROS2 Camera Node!
- Download this file:

wget https://mankiyoon.github.io/courses/csc495/sp26/yolo_detection_node.py

Task 1: View YOLO Results

TODO: Publish annotated image (HINT: Refer Project 1)

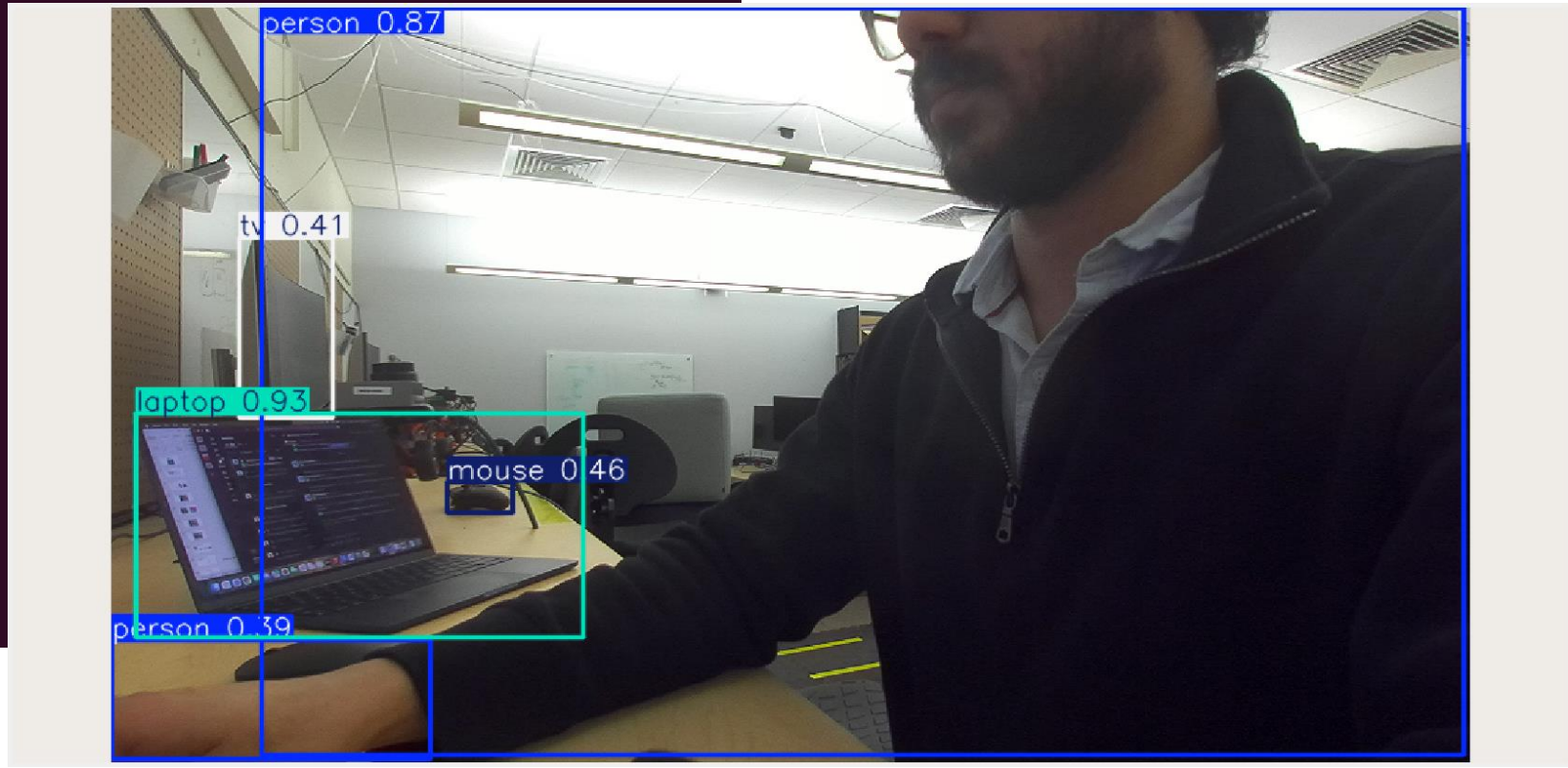
TODO: Publish object count

TODO: View results using RVIZ2

```python
# Callback
def image_callback(self, msg: Image):
    # Convert ROS Image to OpenCV BGR
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

    # Run inference
    results = self.model(frame)
    annotated = results[0].plot()           # draws boxes on the frame
    num_objects = len(results[0].boxes)


    # TODO: Publish annotated image
    ################################
    ## Your code here
    ################################

    # TODO: Publish object count
    ################################
    ## Your code here
    ################################
```

Task 1: View YOLO Results

```
0: 384x640 1 person, 1 stop sign, 262.9ms
Speed: 3.6ms preprocess, 262.9ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 246.8ms
Speed: 4.2ms preprocess, 246.8ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 1 carrot, 242.3ms
Speed: 3.6ms preprocess, 242.3ms inference, 1.5ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 257.6ms
Speed: 3.4ms preprocess, 257.6ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 264.8ms
Speed: 3.6ms preprocess, 264.8ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 278.4ms
Speed: 3.6ms preprocess, 278.4ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 278.9ms
Speed: 3.5ms preprocess, 278.9ms inference, 1.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 240.0ms
Speed: 3.8ms preprocess, 240.0ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 253.0ms
Speed: 3.6ms preprocess, 253.0ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 246.4ms
Speed: 3.5ms preprocess, 246.4ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 259.4ms
Speed: 3.6ms preprocess, 259.4ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 278.0ms
Speed: 3.5ms preprocess, 278.0ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 261.6ms
Speed: 3.9ms preprocess, 261.6ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 256.5ms
Speed: 3.7ms preprocess, 256.5ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 246.9ms
Speed: 3.7ms preprocess, 246.9ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 241.0ms
Speed: 3.5ms preprocess, 241.0ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)
```



Task 2: Identify Stop Sign

TODO: Detect “Stop Sign” and its position and size (width, height)

HINT:

results[0].boxes.cls → class_ids

results[0].boxes.xyxy → top-left-x, top-left-y, bottom-right-x, bottom-right-y

results[0].boxes.names → {class_id: name} dict

```
# Run inference
results = self.model(frame)
annotated = results[0].plot()          # draws boxes on the frame
num_objects = len(results[0].boxes)


# TODO: Publish annotated image
################################
## Your code here
################################

# TODO: Publish object count
################################
## Your code here
################################

# TODO: Get "Stop Sign" Size
# if area >= 300 * 300
# STOP
################################
## Your code here
################################
```

Task 2: Identify Stop Sign

Task 3: Block Throttle

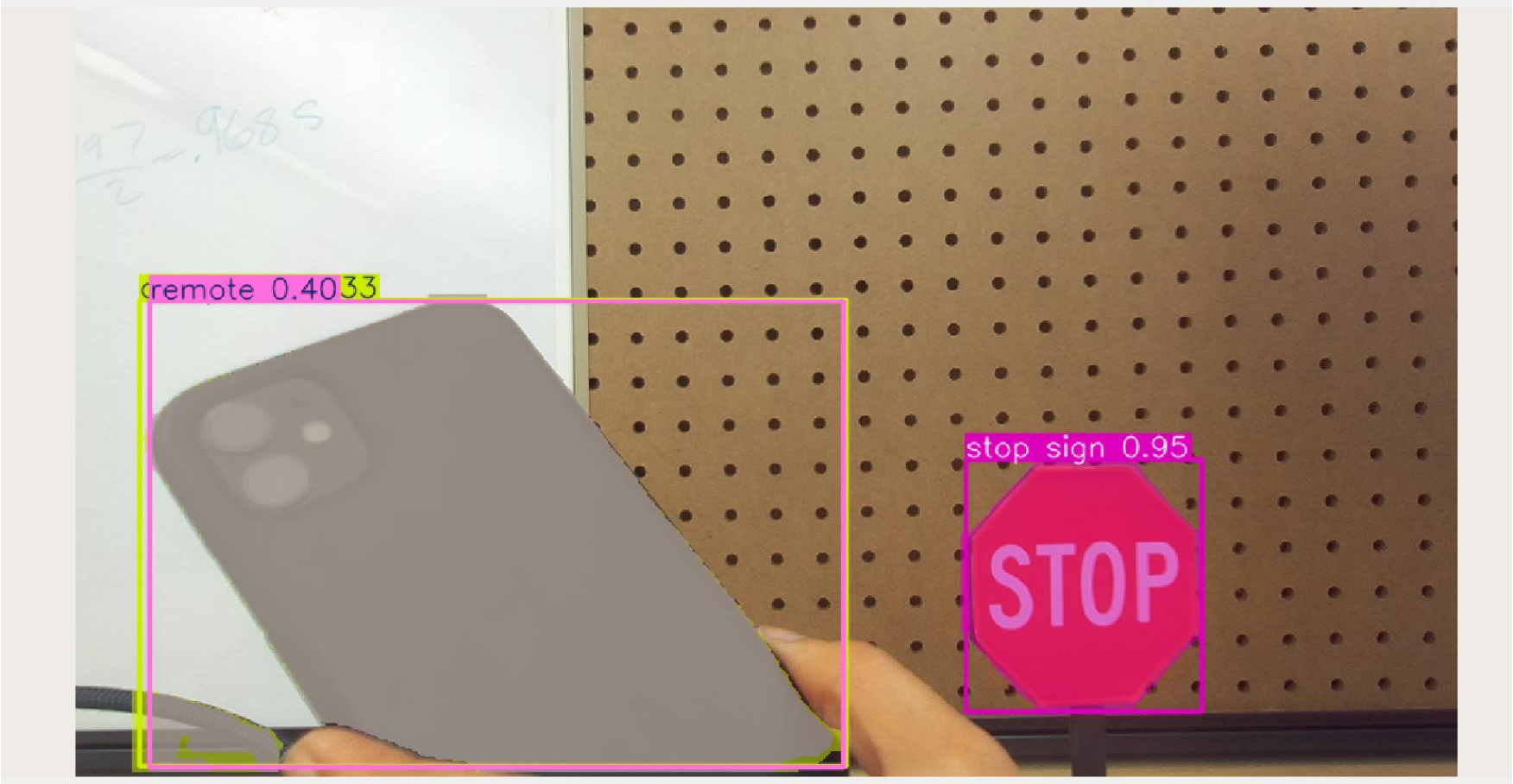
- If there is a “Stop Sign” and Size of “Stop Sign” > 300 * 300
  - Throttle = 0 in driver.py

```
self.sub = self.create_subscription(
    Image, '/zed/zed_node/rgb/color/rect/image', self.image_callback, 10)

self.pub_image = self.create_publisher(Image, '/detections/image', 10)
self.pub_count = self.create_publisher(Int32, '/detections/count', 10)

# TODO: Create topic to send "STOP" Signal
################################
## Your code here
################################

self.get_logger().info('YOLO detection node ready')
```

TODO: Show the TA or Professor!

(Optional) Task 4: YOLO Image Segmentation

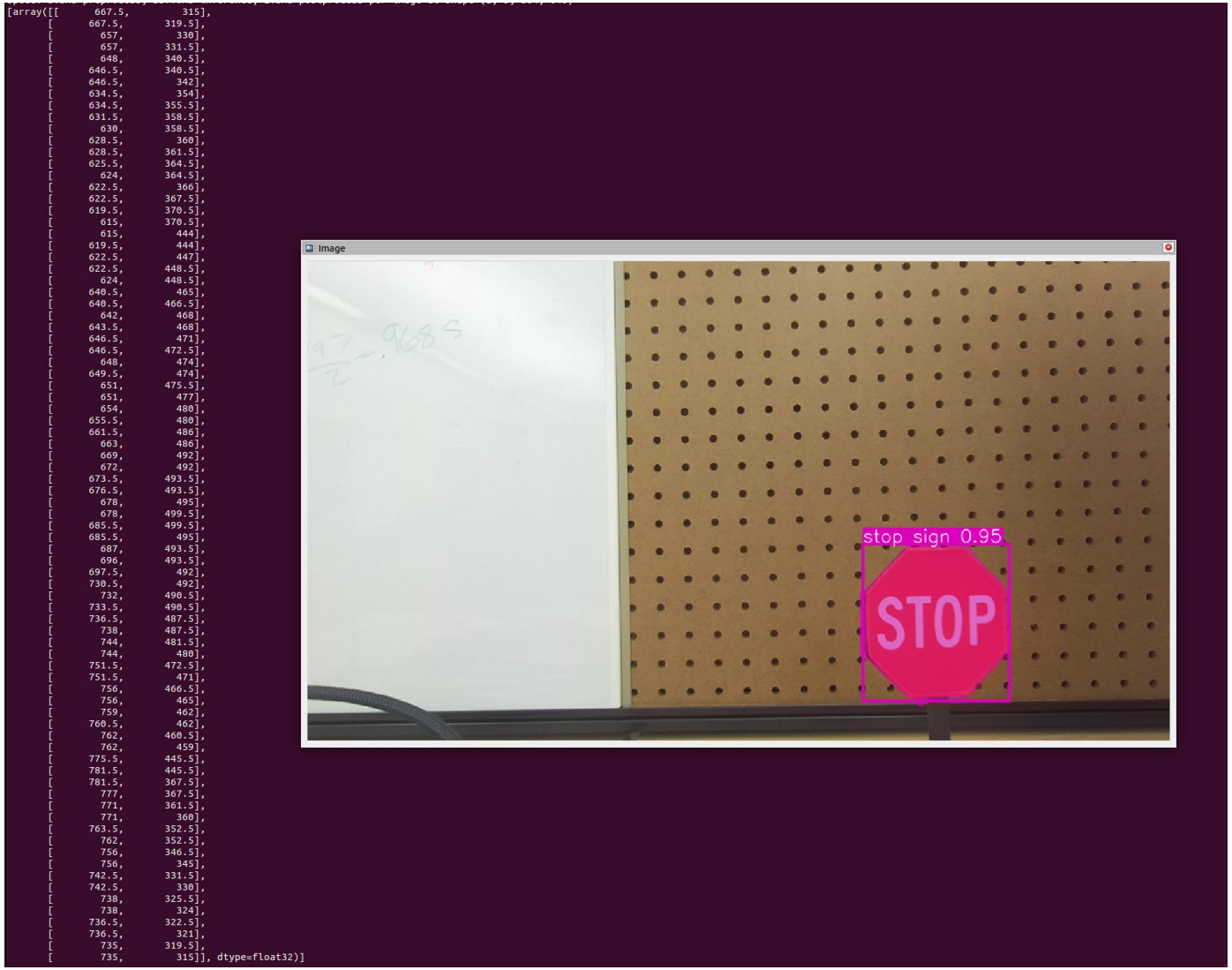
- Change model to ‘yolov8n-seg.pt’



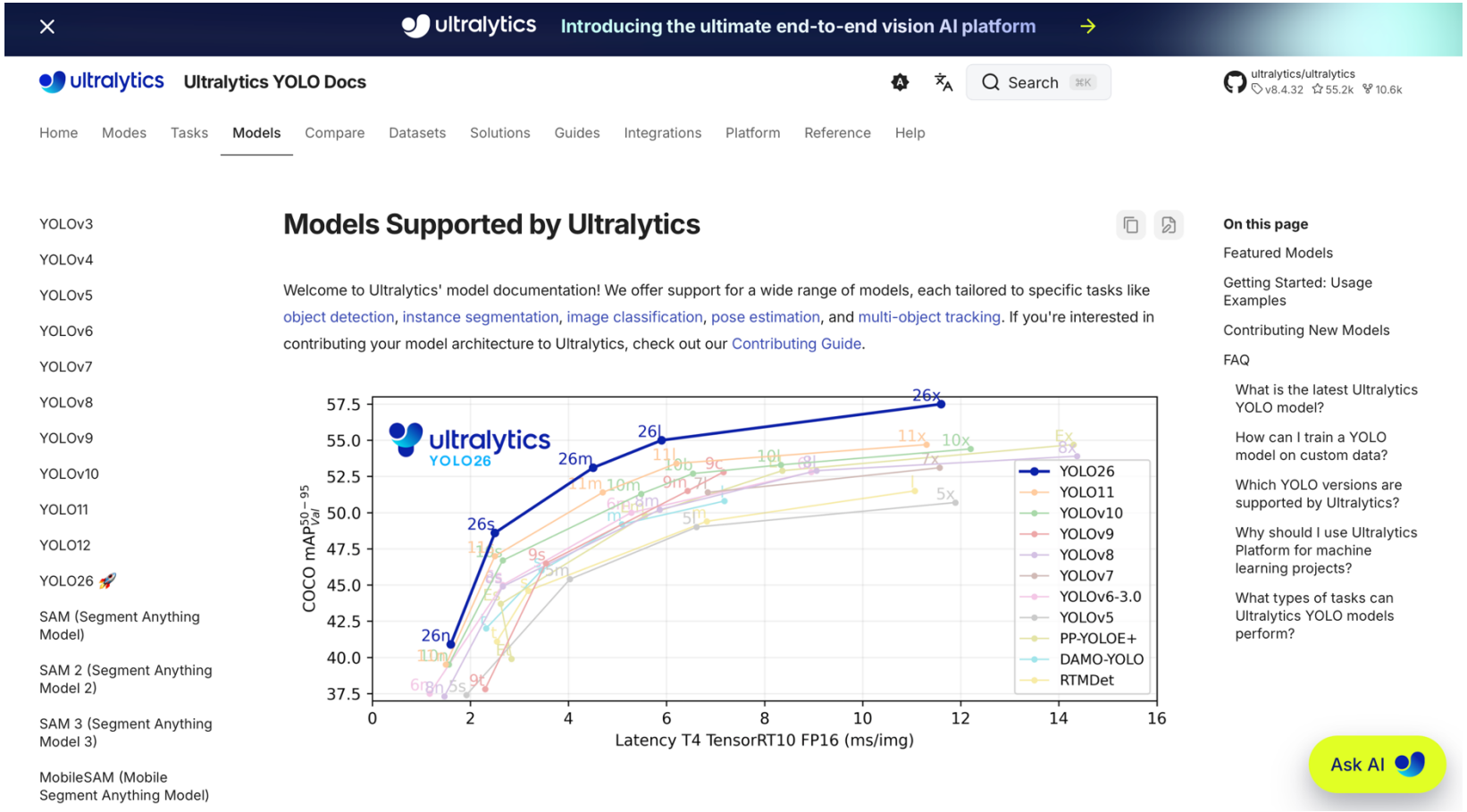
(Optional) Task 4: YOLO Image Segmentation

results[0].masks.xy → class_ids

This prints the detection masks.
What can you do with this?

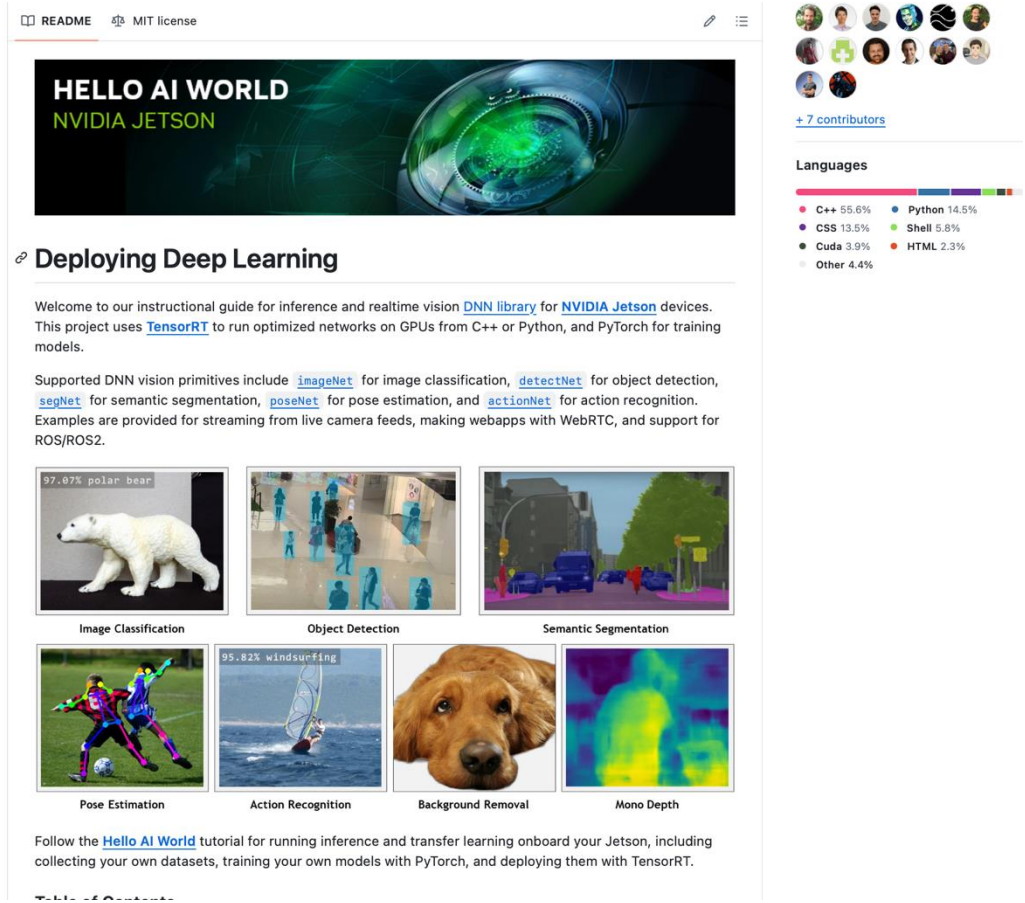


Explore More Resources!



https://docs.ultralytics.com/models/

Explore More Resources!



https://github.com/dusty-nv/jetson-inference